

Lead Full-Stack Engineer Take-Home Test

Overview

Thank you for taking the time to complete this technical exercise.

FGL is building an AI-native software development team to create business-critical products and platforms across cloud products, internal business platforms, workflow automation, integrations, reporting and operational tools.

This exercise is designed to understand how you think, design, build, document and take ownership of a small but realistic product module.

You may choose **one** of the three test options below. Please plan your scope so it can be completed by the submission deadline.

Timeline

The test is being shared on **Monday, 8 June 2026**.

Please submit your completed work by **Sunday, 14 June 2026 at 9:00 AM Bangladesh time (Asia/Dhaka)**.

AI And Tool Usage

You may use any tools you want, including AI-native development tools such as Cursor, GitHub Copilot, Claude Code, ChatGPT, Codex, Windsurf or similar.

You remain responsible for everything you submit. We expect you to review, test and understand all code, architecture and documentation in your submission.

Submission Requirements

Please submit:

1. A **public GitHub repository** link.
2. A working application that can be run locally using Docker.
3. Clear setup instructions in [README.md](#).
4. A short design note explaining your architecture, tradeoffs and assumptions.
5. Any credentials or seed data required for review.

The application should be self-contained. A reviewer should be able to clone the repository and run it without depending on paid external services.

Expected local run experience:

```
git clone <your-public-repo-url>
```

```
cd <repo>
```

```
docker compose up --build
```

If you use external APIs, provide mocks, stubs or documented local alternatives.

What We Are Looking For

We are not looking for a large system. We are looking for judgment.

We will evaluate:

- Product thinking and user flow.
- Backend/API design.
- Frontend usability and state handling.
- Data modeling and persistence.
- Error handling and edge cases.
- Security and access-control awareness.
- Testing approach.
- Docker/self-contained deployment.
- Code readability and maintainability.
- Documentation and assumptions.
- How responsibly you use AI-assisted development, if used.

Option 1: ERP Workflow Module

Context

FGL has business teams that often need software to reduce manual coordination between Sales, Accounts, Operations and Management.

Design and build a small ERP-style workflow module for an internal business process.

The exact process is up to you, but it should involve at least:

- A request created by one role or department.
- Review or approval by another role.
- A final business outcome, such as an invoice, purchase request, payment request, procurement approval, HR approval, delivery approval or similar.
- Status tracking and auditability.

Example Direction

One possible flow:

Sales submits a billing request. Accounts reviews it. If approved, an invoice is created and becomes visible in a dashboard.

You may choose this flow or design a different ERP/business workflow if you think another one demonstrates your judgment better.

Minimum Expectations

Your solution should include:

- At least 2-3 user roles.
- A clear workflow with statuses.
- Create, view, approve/reject and update actions where appropriate.
- Basic dashboard/list views.
- Audit trail or activity history.
- Sensible data model.
- Basic validation and error states.
- Seed data for review.

Space For Creativity

You may add:

- Role-based access control.
- Comments or attachments.
- Notifications or simulated notifications.
- Reporting/summary metrics.
- ERPNext/Frappe-style integration mock.
- Accounting/invoice object model.
- Approval rules.
- Exportable reports.
- AI-assisted draft generation or workflow explanation.

Option 2: PICO Self-Service Cloud Module

Context

PICO is FGL's cloud platform. The target experience is a modern self-service journey where a customer can discover services, sign up, choose a resource, understand pricing, provision infrastructure, view usage and manage billing without manual support.

Design and build a small self-service cloud module.

The exact scope is up to you. You do not need to integrate with a real cloud provider. You may mock infrastructure APIs.

Example Direction

One possible flow:

A customer signs up, selects a VM package, sees estimated monthly pricing, provisions a simulated VM, views status/usage, receives an invoice and marks payment as completed.

You may choose this flow or design a different cloud self-service flow if you think another one demonstrates your judgment better.

Minimum Expectations

Your solution should include:

- Customer-facing flow.
- Service/package selection.
- Pricing or cost estimate.
- Simulated provisioning state machine.
- Resource list/detail view.
- Billing or invoice view.
- Clear error/loading/empty states.
- Mocked infrastructure API or service layer.
- Seed data for review.

Space For Creativity

You may add:

- OpenStack/Mirantis-style API mocks.

- VM, storage, network or IP allocation concepts.
- Usage metering.
- Payment simulation.
- Service health/status page.
- SLA or incident status.
- RBAC/API keys.
- Audit logs.
- Terraform-like provisioning plan preview.
- AI-assisted support/chat or configuration explanation.

Option 3: Operations Intelligence Module

Context

FGL's future software team will build products that help teams see what is happening across systems, workflows and operations. Many internal teams struggle with fragmented information, unclear status, manual reporting and delayed escalation.

Design and build a small operations intelligence module that helps a team monitor, understand and act on operational data.

The domain is up to you. Choose something practical: service health, workflow bottlenecks, customer support, procurement status, billing issues, deployment health, sales pipeline, field operations, incident tracking or another operational area.

Example Direction

One possible flow:

An operations dashboard shows incoming issues from multiple sources, groups them by severity, lets a user assign owners, tracks resolution status and produces a daily summary.

You may choose this flow or design a different operational module if you think another one demonstrates your judgment better.

Minimum Expectations

Your solution should include:

- A dashboard or operational overview.
- Multiple records/events/items with different statuses.
- Filtering or grouping.

- Detail view for one item.
- Ability to take an action, such as assign, resolve, escalate, comment or acknowledge.
- Basic summary metrics.
- Seed data or simulated event generation.
- Clear explanation of the operational problem you chose.

Space For Creativity

You may add:

- AI-generated operational summary.
- SLA breach detection.
- Incident timeline.
- Alert routing.
- Mock webhook ingestion.
- CSV import/export.
- Role-based views.
- Trend charts.
- Root-cause notes.
- Suggested next action.

Technical Freedom

Use any stack you prefer.

Examples:

- Backend: Python, Go, Node.js/TypeScript, Java, .NET, PHP or others.
- Frontend: React, Next.js, Vue, Angular, Svelte or server-rendered UI.
- Database: PostgreSQL, MySQL, SQLite, MongoDB or similar.
- Deployment: Docker Compose is preferred.

Choose tools that let you demonstrate product and engineering judgment within the time limit.

Documentation Expectations

Your `README.md` should include:

- What you built.
- Which option you selected.
- How to run it.
- Demo credentials, if any.
- Key user flows.

- Architecture overview.
- Data model overview.
- Known limitations.
- What you would improve with more time.
- AI tools used, if any, and how you reviewed the output.

Review Rubric

Area	Weight	What We Look For
Product/user flow	20%	Clear workflow, useful screens, sensible states and real user empathy.
Backend/API/data model	20%	Clean domain model, APIs/service boundaries, validation and persistence.
Frontend implementation	15%	Usable UI, state handling, loading/error/empty states and responsiveness.
Ownership and engineering judgment	15%	Good tradeoffs, readable code, maintainable structure and scoped delivery.
Reliability/security/testing	15%	Error handling, access-control thinking, tests or testable design.
Docker/deployment/documentation	10%	Reviewer can run it easily and understand decisions.
AI-native development reflection	5%	Responsible AI usage, review process and clear ownership of generated work.

Notes

- Do not spend time building authentication perfectly unless it is central to your design. Simple mocked users/roles are fine.
- Do not use paid services or require private credentials.
- Do not overbuild. A thoughtful, complete small module is better than a large unfinished system.

- We value clarity, ownership and judgment more than visual polish alone.